JavaScript Execution Context & Call Stack

Notes based on "Chai aur Code" Tutorial

Whenever JavaScript executes your code, it runs in a specific environment called an **Execution Context**. Understanding how this context is created and managed is the fundamental key to understanding variable hoisting, scope, and the call stack.

> **Global Execution Context (GEC):** When a JS script starts, a GEC is created first and assigned to `this`.
>
> *Note:* The GEC differs based on the environment. In a browser, `this` equals the `window` object. In environments like Node.js, it equals an empty object `{}`.

Types of Execution Contexts

- **Global Execution Context:** The default context where code runs initially.
- **Function Execution Context:** Created dynamically whenever a function is called or invoked.
- **Eval Execution Context:** Associated with the `eval()` function (rarely used in modern JS, but present in internal engines like Mongoose).

The Two Phases of Execution

JavaScript code is always executed in two distinct phases:

1. **Memory Creation Phase (or Creation Phase):** The JS engine scans the code and allocates memory space for variables and functions. *No code is executed here.* Variables are initialized with the special value `undefined`, while functions hold their entire code definitions.
2. **Execution Phase:** The engine reads the code line by line again. This time, it assigns real values to the variables and executes function calls.

Example Code for Walkthrough

We will trace the execution of the following standard snippet to understand how the internal engine processes it:

```
      let val1 = 10; let val2 = 5; function addNum(num1, num2) { let total = num1 + num2; return total; } let result1 = addNum(val1, val2); let result2 = addNum(10, 2);
```

Step 1: Memory Creation Phase

The moment the code is fed to the engine, the Global Execution Context is pushed into the Call Stack. Then, the Memory Phase begins. The engine scans the snippet and assigns memory:

- `val1` is allocated memory and set to `undefined`.
- `val2` is allocated memory and set to `undefined`.
- `addNum` is allocated memory and stores the *entire function definition*.
- `result1` and `result2` are allocated memory and set to `undefined`.

Memory Creation Phase (Global)
`val1 -> undefined`
`val2 -> undefined`
`addNum -> [Function Definition]`
`result1, result2 -> undefined`

Step 2: Execution Phase

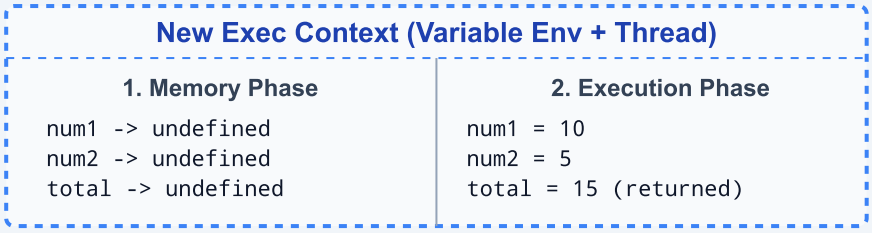
Now, the engine reads the code line by line to execute it. Values are assigned to the variables.

- `val1 = 10` (replaces undefined)
- `val2 = 5` (replaces undefined)
- The engine skips over the `addNum` definition because it is just a declaration.
- At `let result1 = addNum(val1, val2)`, a function is invoked. **This is a critical moment.**

> **New Execution Context:** Whenever a function is executed, a brand new *"Variable Environment + Execution Thread"* is created. This creates a mini-sandbox. Inside this sandbox, the Two Phases (Memory and Execution) happen all over again just for that function!

Inside the addNum Execution Context

For `result1 = addNum(10, 5)`, the new sandbox undergoes its own phases:



Once `return total;` is executed, the value (15) is passed back to the parent (Global Execution Context) to be stored in `result1`. Immediately after returning the value, **the function's execution context is deleted** to free up memory.

This entire process repeats identically for `result2 = addNum(10, 2)`.

The Call Stack

The JS engine manages these multiple execution contexts using the **Call Stack**. The Call Stack operates on a **LIFO (Last In, First Out)** principle.



The Global context stays at the bottom. When addNum is called, it is pushed onto the stack. When it returns a value, it is popped off the stack. If addNum called another function, that inner function would be pushed on top of it, executing first (LIFO).